

Connection:

We used GCP to host our sql server and build our database. The DDL for constructing all tables is visible in the DatabaseDesign.md file. All data was uploaded in the form of CSVs visible in the Dataset subdirectory of this repo.

```
noahaz68@cloudshell:~ (cs411-491905)$ gcloud sql connect cs411-blue --user=root
Starting Cloud SQL Proxy: [/usr/bin/cloud-sql-proxy cs411-491905:us-central1:cs411-blue --port 9470]
2026/03/31 14:30:41 Authorizing with Application Default Credentials
2026/03/31 14:30:41 [cs411-491905:us-central1:cs411-blue] Listening on 127.0.0.1:9470
2026/03/31 14:30:41 The proxy has started successfully and is ready for new connections!
Connecting to database with SQL user [root].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14185
Server version: 8.0.44-google (Google)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use flights;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

Tables (1000 rows in at least 3 tables):

```
mysql> SELECT 'dim_wac' AS table_name, COUNT(*) AS row_count FROM dim_wac
-> UNION ALL
-> SELECT 'dim_date', COUNT(*) FROM dim_date
-> UNION ALL
-> SELECT 'dim_airport', COUNT(*) FROM dim_airport
-> UNION ALL
-> SELECT 'dim_carrier', COUNT(*) FROM dim_carrier
-> UNION ALL
-> SELECT 'dim_delay_type', COUNT(*) FROM dim_delay_type
-> UNION ALL
-> SELECT 'fact_flight', COUNT(*) FROM fact_flight
-> UNION ALL
-> SELECT 'fact_flight_delay', COUNT(*) FROM fact_flight_delay
-> UNION ALL
-> SELECT 'saved_preset', COUNT(*) FROM saved_preset;
+-----+-----+
| table_name | row_count |
+-----+-----+
| dim_wac    |         335 |
| dim_date   |          31 |
| dim_airport |        6820 |
| dim_carrier |        1772 |
| dim_delay_type |          5 |
| fact_flight |       599013 |
| fact_flight_delay |     173575 |
| saved_preset |           0 |
+-----+-----+
8 rows in set (2.19 sec)
```

Advanced Queries

Query 1: Top 15 carriers by average arrival delay, only for carriers with above-average delay (these carriers have an average arrival delay higher than the overall average across all flights in the dataset, use in app to see the “worst of the worst” carriers)

Concepts: JOIN + GROUP BY + Subquery

```
SELECT c.carrier, c.carrier_name,
       COUNT(*) AS total_flights,
       ROUND(AVG(f.arr_delay), 2) AS avg_arr_delay
FROM fact_flight f
JOIN dim_carrier c ON f.mkt_carrier_airline_id = c.airline_id
WHERE f.arr_delay IS NOT NULL
GROUP BY c.carrier, c.carrier_name
HAVING AVG(f.arr_delay) > (
    SELECT AVG(arr_delay) FROM fact_flight WHERE arr_delay IS NOT NULL
)
ORDER BY avg_arr_delay DESC
LIMIT 15;
```

Output: (ONLY 7 CARRIERS ABOVE AVG DELAY FOR FLIGHTS IN GENERAL)

```
mysql> SELECT c.carrier, c.carrier_name,
->          COUNT(*) AS total_flights,
->          ROUND(AVG(f.arr_delay), 2) AS avg_arr_delay
-> FROM fact_flight f
-> JOIN dim_carrier c ON f.mkt_carrier_airline_id = c.airline_id
-> WHERE f.arr_delay IS NOT NULL
-> GROUP BY c.carrier, c.carrier_name
-> HAVING AVG(f.arr_delay) > (
->     SELECT AVG(arr_delay) FROM fact_flight WHERE arr_delay IS NOT NULL
-> )
-> ORDER BY avg_arr_delay DESC
-> LIMIT 15;
```

carrier	carrier_name	total_flights	avg_arr_delay
F9	Frontier Airlines Inc.	15526	10.53
G4	Allegiant Air	9345	9.40
AA	American Airlines Inc.	155779	6.04
HA	Hawaiian Airlines Inc.	6690	5.90
B6	JetBlue Airways	17918	5.69
DL	Delta Air Lines Inc.	123672	4.46
UA	United Air Lines Inc.	115422	4.41

Query 2: Airports that are top-15 busiest as either origin or destination (use in app to see which airports are the busiest, their flight counts, separated by role in terms of origin and destination)

Concepts: JOIN + GROUP BY + SET Operator (UNION)

```
(SELECT a.airport, a.display_airport_name,
       COUNT(*) AS flight_count, 'Origin' AS role
FROM fact_flight f
JOIN dim_airport a ON f.origin_airport_id = a.airport_id
GROUP BY a.airport, a.display_airport_name
ORDER BY flight_count DESC
LIMIT 15)
UNION
(SELECT a.airport, a.display_airport_name,
       COUNT(*) AS flight_count, 'Destination' AS role
FROM fact_flight f
JOIN dim_airport a ON f.dest_airport_id = a.airport_id
GROUP BY a.airport, a.display_airport_name
ORDER BY flight_count DESC
LIMIT 15);
```

Output:

```

-> JOIN dim_airport a ON f.origin_airport_id = a.airport_id
-> GROUP BY a.airport, a.display_airport_name
-> ORDER BY flight_count DESC
-> LIMIT 15)
-> UNION
-> (SELECT a.airport, a.display_airport_name,
->      COUNT(*) AS flight_count, 'Destination' AS role
-> FROM fact_flight f
-> JOIN dim_airport a ON f.dest_airport_id = a.airport_id
-> GROUP BY a.airport, a.display_airport_name
-> ORDER BY flight_count DESC
-> LIMIT 15);

```

airport	display_airport_name	flight_count	role
ATL	Hartsfield-Jackson Atlanta International	27073	Origin
DFW	Dallas/Fort Worth International	25248	Origin
ORD	Chicago O'Hare International	25058	Origin
DEN	Denver International	24732	Origin
CLT	Charlotte Douglas International	20190	Origin
PHX	Phoenix Sky Harbor International	16031	Origin
LAX	Los Angeles International	15596	Origin
LAS	Harry Reid International	15196	Origin
SEA	Seattle/Tacoma International	13618	Origin
IAH	George Bush Intercontinental/Houston	13609	Origin
LGA	LaGuardia	13254	Origin
MCO	Orlando International	13058	Origin
DCA	Ronald Reagan Washington National	12152	Origin
EWR	Newark Liberty International	11492	Origin
SFO	San Francisco International	11168	Origin
ATL	Hartsfield-Jackson Atlanta International	27083	Destination
DFW	Dallas/Fort Worth International	25248	Destination
ORD	Chicago O'Hare International	25057	Destination
DEN	Denver International	24740	Destination
CLT	Charlotte Douglas International	20197	Destination
PHX	Phoenix Sky Harbor International	16024	Destination
LAX	Los Angeles International	15582	Destination
LAS	Harry Reid International	15194	Destination
SEA	Seattle/Tacoma International	13625	Destination
IAH	George Bush Intercontinental/Houston	13612	Destination
LGA	LaGuardia	13242	Destination
MCO	Orlando International	13052	Destination
DCA	Ronald Reagan Washington National	12159	Destination
EWR	Newark Liberty International	11486	Destination
SFO	San Francisco International	11161	Destination

Query 3: Top 10 routes by total delay minutes broken down by delay type (use in app to see the routes with worst weather, ATS control, carriers, etc.)

Concepts: JOIN (4 relations) + GROUP BY + Subquery

```
SELECT origin.airport AS origin_code,
       dest.airport AS dest_code,
       dt.delay_type_name,
       ROUND(SUM(fd.delay_minutes), 2) AS total_delay_minutes,
       COUNT(*) AS delay_occurrences
FROM fact_flight_delay fd
JOIN fact_flight f ON fd.flight_id = f.flight_id
JOIN dim_airport origin ON f.origin_airport_id = origin.airport_id
JOIN dim_airport dest ON f.dest_airport_id = dest.airport_id
JOIN dim_delay_type dt ON fd.delay_type_id = dt.delay_type_id
WHERE (f.origin_airport_id, f.dest_airport_id) IN (
  SELECT sub.origin_airport_id, sub.dest_airport_id
  FROM (
    SELECT origin_airport_id, dest_airport_id,
         COUNT(*) AS delay_count
    FROM fact_flight
    WHERE arr_del15 = 1
    GROUP BY origin_airport_id, dest_airport_id
    ORDER BY delay_count DESC
    LIMIT 10
  ) sub
)
GROUP BY origin.airport, dest.airport, dt.delay_type_name
ORDER BY total_delay_minutes DESC
LIMIT 15;
```

Output:

origin_code	dest_code	delay_type_name	total_delay_minutes	delay_occurrences
ATL	MCO	Carrier	5729.00	120
MCO	ATL	Late Aircraft	5625.00	92
FLL	ATL	Carrier	5444.00	77
FLL	ATL	Late Aircraft	4618.00	80
ORD	LGA	NAS	4568.00	123
ATL	FLL	Carrier	4157.00	107
MCO	ATL	NAS	4097.00	80
ATL	MCO	Late Aircraft	3941.00	78
LAX	LAS	NAS	3757.00	105
DEN	SLC	Carrier	3718.00	104
DEN	LAS	NAS	3575.00	107
LAX	LAS	Late Aircraft	3560.00	80
MIA	LGA	Late Aircraft	3554.00	74
ORD	LGA	Carrier	3548.00	88
MIA	LGA	Carrier	3482.00	90

INDEXING:

NOTE: We had to evaluate some query in an environment where previously created indexes were still present, since some indexes couldn't be removed due to MySQL foreign key behavior. The analysis focuses on the optimizer costs of the tested configurations in this environment.

QUERY 1

Baseline:

```
mysql> EXPLAIN ANALYZE
-> SELECT c.carrier, c.carrier_name,
->        COUNT(*) AS total_flights,
->        ROUND(AVG(f.arr_delay), 2) AS avg_arr_delay
-> FROM fact_flight f FORCE INDEX (PRIMARY)
-> JOIN dim_carrier c ON f.mkt_carrier_airline_id = c.airline_id
-> WHERE f.arr_delay IS NOT NULL
-> GROUP BY c.carrier, c.carrier_name
-> HAVING AVG(f.arr_delay) > (
->   SELECT AVG(arr_delay) FROM fact_flight WHERE arr_delay IS NOT NULL
-> )
-> ORDER BY avg_arr_delay DESC
-> LIMIT 15;

+-----+
| EXPLAIN |
+-----+

+-----+
| Limit: 15 row(s) (actual time=2830..2830 rows=7 loops=1)
| Sort: avg_arr_delay DESC (actual time=2830..2830 rows=7 loops=1)
| Filter: (??? > (select #2)) (actual time=2830..2830 rows=7 loops=1)
| Table scan on <temporary> (actual time=2053..2053 rows=10 loops=1)
| Aggregate using temporary table (actual time=2053..2053 rows=10 loops=1)
| Nested loop inner join (cost=1.25e+6 rows=536302) (actual time=104..1138 rows=599013 loops=1)
| Filter: ((f.arr_delay is not null) and (f.mkt_carrier_airline_id is not null)) (cost=655481 rows=536302) (actual time=82.4..857 rows=599013 loops=1)
| Table scan on f (cost=655481 rows=536302) (actual time=82.4..782 rows=599013 loops=1)
| Single-row index lookup on c using PRIMARY (airline_id=f.mkt_carrier_airline_id) (cost=1 rows=1) (actual time=199e-6..236e-6 rows=1 loops=599013)
| Select #2 (subquery in condition; run only once)
| Aggregate: avg(fact_flight.arr_delay) (cost=89857 rows=1) (actual time=777..777 rows=1 loops=1)
| Filter: (fact_flight.arr_delay is not null) (cost=60062 rows=297945) (actual time=2.11..675 rows=599013 loops=1)
| Covering index range scan on fact_flight using idx_q1_designC_arrdelay over (NULL < arr_delay) (cost=60062 rows=297945) (actual time=2.11..613 rows=599013 loops=1)
+-----+

1 row in set (3.10 sec)
```

Index Design A: Composite index on fact_flight(mkt_carrier_airline_id, arr_delay)

```
mysql> EXPLAIN ANALYZE
-> SELECT c.carrier, c.carrier_name,
->        COUNT(*) AS total_flights,
->        ROUND(AVG(f.arr_delay), 2) AS avg_arr_delay
-> FROM fact_flight f USE INDEX (idx_q1_designA_carrier_arrdelay)
-> JOIN dim_carrier c ON f.mkt_carrier_airline_id = c.airline_id
-> WHERE f.arr_delay IS NOT NULL
-> GROUP BY c.carrier, c.carrier_name
-> HAVING AVG(f.arr_delay) > (
->   SELECT AVG(arr_delay) FROM fact_flight WHERE arr_delay IS NOT NULL
-> )
-> ORDER BY avg_arr_delay DESC
-> LIMIT 15;

+-----+
| EXPLAIN |
+-----+

+-----+
| Limit: 15 row(s) (actual time=3729..3729 rows=7 loops=1)
| Sort: avg_arr_delay DESC (actual time=3729..3729 rows=7 loops=1)
| Filter: (??? > (select #2)) (actual time=3729..3729 rows=7 loops=1)
| Table scan on <temporary> (actual time=3028..3028 rows=10 loops=1)
| Aggregate using temporary table (actual time=3028..3028 rows=10 loops=1)
| Nested loop inner join (cost=518195 rows=536302) (actual time=95.6..1413 rows=599013 loops=1)
| Filter: ((f.arr_delay is not null) and (f.mkt_carrier_airline_id is not null)) (cost=62339 rows=536302) (actual time=95.6..1151 rows=599013 loops=1)
| Covering index scan on f using idx_q1_designA_carrier_arrdelay (cost=62339 rows=595891) (actual time=95.6..1066 rows=599013 loops=1)
| Single-row index lookup on c using PRIMARY (airline_id=f.mkt_carrier_airline_id) (cost=0.75 rows=1) (actual time=166e-6..204e-6 rows=1 loops=599013)
| Select #2 (subquery in condition; run only once)
| Aggregate: avg(fact_flight.arr_delay) (cost=89857 rows=1) (actual time=701..701 rows=1 loops=1)
| Filter: (fact_flight.arr_delay is not null) (cost=60062 rows=297945) (actual time=0.282..607 rows=599013 loops=1)
| Covering index range scan on fact_flight using idx_q1_designC_arrdelay over (NULL < arr_delay) (cost=60062 rows=297945) (actual time=0.281..549 rows=599013 loops=1)
+-----+

1 row in set (3.90 sec)
```

Index Design B: composite index on fact_flight(arr_delay, mkt_carrier_airline_id)

```
mysql> EXPLAIN ANALYZE
-> SELECT c.carrier, c.carrier_name,
->        COUNT(*) AS total_flights,
->        ROUND(AVG(f.arr_delay), 2) AS avg_arr_delay
-> FROM fact_flight f USE INDEX (idx_qt_designc_arrraydelay_carriers)
-> JOIN dim_carrier c ON f.mkt_carrier_id = c.airline_id
-> WHERE f.arr_delay IS NOT NULL
-> GROUP BY c.carrier, c.carrier_name
-> HAVING AVG(f.arr_delay) > 1
-> SELECT AVG(arr_delay) FROM fact_flight WHERE arr_delay IS NOT NULL
->
-> ORDER BY avg_arr_delay DESC
-> LIMIT 10;

+-----+
| EXPLAIN |
+-----+

+-----+
|      |
+-----+

+-----+
| Limit: 10 row(s)   (actual time=3343..3348 rows=7 loops=1) |
| Sort: avg_arr_delay DESC   (actual time=3343..3348 rows=7 loops=1) |
| Filter: (f.f > {select f2})   (actual time=3343..3348 rows=7 loops=1) |
|   -> Filter: (f.f > {select f2})   (actual time=3343..3348 rows=7 loops=1) |
|     -> Table scan on temporary:   (actual time=2681..2681 rows=10 loops=1) |
|       -> Aggregate using temporary table   (actual time=2681..2681 rows=10 loops=1) |
|         -> Nested loop inner join   (cost=0.197 rows=397945) (actual time=0.478..1782 rows=599013 loops=1) |
|           -> Filter: ((f.arr_delay is not null) and (f.mkt_carrier_airline_id is not null))   (cost=0.0245 rows=297945) (actual time=0.463..1478 rows=599013 loops=1) |
|             -> Covering index range scan on f using idx_qt_designc_arrraydelay_carriers over (NULL < arr_delay)   (cost=0.0245 rows=297945) (actual time=0.461..1464 rows=599013 loops=1) |
|               -> Single-row index lookup on c using PRIMARY (airline_id=f.mkt_carrier_airline_id)   (cost=0.75 rows=1) (actual time=181e-6..219e-6 rows=1 loops=599013) |
|                 -> Select #2 (subquery in condition row only once) |
|                   -> Aggregate using fact_flight.arr_delay   (cost=0.8857 rows=1) (actual time=662..662 rows=1 loops=1) |
|                     -> Filter: (fact_flight.arr_delay is not null)   (cost=0.0062 rows=297945) (actual time=0.3..561 rows=599013 loops=1) |
|                       -> Covering index range scan on fact_flight using idx_qt_designc_arrraydelay over (NULL < arr_delay)   (cost=0.0062 rows=297945) (actual time=0.299..501 rows=599013 loops=1) |
+-----+

1 row in set (3.46 sec)
```

Index Design C: single-column index on fact_flight(arr_delay)

[illegible]

Analysis:

For Query 1, the indexing configurations we tested didn't improve the optimizer cost that much compared to the baseline execution. The baseline configuration had a dominant nested loop join cost of 1.25e+6, but the three tested designs had corresponding costs of 518,195 for Design A, 313,497 for Design B, and 315,775 for Design C. Among the three added index designs, Design B, which was the composite index on (arr_delay, mkt_carrier_airline_id), produced the lowest cost, but even with the lower join cost, the query still required a temporary table for aggregation and a final sort on avg_arr_delay, so the indexing benefit was limited.

This shows that Query 1 is driven mainly by large-scale aggregation and not a highly selective indexed lookup. The query still needs to process a very large portion of `fact_flight`, join those rows to `dim_carrier`, and compute grouped averages before applying the `HAVING` clause. The scalar subquery computing the overall average arrival delay consistently benefited from the `arr_delay` index, but the main grouped query was still expensive even with indexing. The best tested indexing design for Query 1 was Design B, which is the one we chose as our final design, but the benefit wasn't that great because aggregation and sorting were still the dominant costs.

Baseline:

```

mysql> EXPLAIN ANALYZE
-> SELECT *
-> FROM (
->   (SELECT a.airport, a.display_airport_name,
->        COUNT(*) AS flight_count, 'Origin' AS role
->    FROM fact_flight f
->   JOIN dim_airport a ON f.origin_airport_id = a.airport_id
->  GROUP BY a.airport, a.display_airport_name
-> ORDER BY flight_count DESC
-> LIMIT 15)
-> UNION
->   (SELECT a.airport, a.display_airport_name,
->        COUNT(*) AS flight_count, 'Destination' AS role
->    FROM fact_flight f
->   JOIN dim_airport a ON f.dest_airport_id = a.airport_id
->  GROUP BY a.airport, a.display_airport_name
-> ORDER BY flight_count DESC
-> LIMIT 15)
->) q;

+-----+
| EXPLAIN |
+-----+

+-----+
| Table scan on q (cost=2.5..2.5 rows=0) (actual time=8598..8598 rows=30 loops=1)
-> Union materializes with deduplication (cost=0..0 rows=0) (actual time=8598..8598 rows=30 loops=1)
-> Limit: 15 row(s) (actual time=4509..4509 rows=15 loops=1)
-> Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4509..4509 rows=15 loops=1)
-> Table scan on <temporary> (actual time=4509..4509 rows=352 loops=1)
-> Aggregate using temporary table (actual time=4509..4509 rows=352 loops=1)
-> Nested loop inner join (cost=717870 rows=595891) (actual time=216..1768 rows=599013 loops=1)
-> Filter: (f.origin_airport_id is not null) (cost=62390 rows=595891) (actual time=171..1350 rows=599013 loops=1)
-> Covering index scan on f using idx_q2_designnc_origin_only (cost=62390 rows=595891) (actual time=171..1292 rows=599013 loops=1)
-> Single-row index lookup on a using PRIMARY (airport_id=f.origin_airport_id) (cost=1 row=1) (actual time=417e-6..455e-6 rows=1 loops=599013)
-> Limit: 15 row(s) (actual time=4088..4088 rows=15 loops=1)
-> Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4088..4088 rows=15 loops=1)
-> Table scan on <temporary> (actual time=4088..4088 rows=352 loops=1)
-> Aggregate using temporary table (actual time=4088..4088 rows=352 loops=1)
-> Nested loop inner join (cost=717870 rows=595891) (actual time=7.88..1315 rows=599013 loops=1)
-> Filter: (f.dest_airport_id is not null) (cost=62390 rows=595891) (actual time=6.88..979 rows=599013 loops=1)
-> Covering index scan on f using idx_q2_designnc_dest_only (cost=62390 rows=595891) (actual time=6.88..916 rows=599013 loops=1)
-> Single-row index lookup on a using PRIMARY (airport_id=f.dest_airport_id) (cost=1 row=1) (actual time=284e-6..322e-6 rows=1 loops=599013)

+-----+
1 row in set (8.65 sec)
```

Design A: composite index on fact_flight(origin_airport_id, dest_airport_id)

```
mysql> EXPLAIN ANALYZE
-> SELECT *
-> FROM (
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Origin' AS role
->    FROM fact_flight f USE INDEX (idx_g2_designa_origin_dest)
->    JOIN dim_airport a ON f.origin_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
-> UNION
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Destination' AS role
->    FROM fact_flight f USE INDEX (idx_g2_designa_origin_dest)
->    JOIN dim_airport a ON f.dest_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
-> ) q;

+-----+
| EXPLAIN |
+-----+
+-----+
|> Table scan on q (cost=2.5..2.5 rows=0) (actual time=9055..9055 rows=30 loops=1)
|> Union Materialize with deduplication (cost=0..0 rows=0) (actual time=9055..9055 rows=30 loops=1)
|> Limit: 15 row(s) (actual time=4178..4178 rows=15 loops=1)
|> Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4178..4178 rows=15 loops=1)
|> Table scan on <temporary> (actual time=4178..4178 rows=352 loops=1)
|> Aggregate using temporary table (actual time=4178..4178 rows=352 loops=1)
|> Nested loop inner join (cost=-398993 rows=595891) (actual time=161..1373 rows=599013 loops=1)
|> Filter: (f.origin_airport_id is not null) (cost=62740 rows=595891) (actual time=127..397 rows=599013 loops=1)
|> Covering index scan on f using idx_g2_designa_origin_dest (cost=62740 rows=595891) (actual time=127..934 rows=599013 loops=1)
|> Single-row index lookup on a using PRIMARY (airport_id=f.origin_airport_id) (cost=0.464 rows=1) (actual time=331e-6..369e-6 rows=1 loops=599013)
|> Limit: 15 row(s) (actual time=4877..4877 rows=15 loops=1)
|> Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4877..4877 rows=15 loops=1)
|> Table scan on <temporary> (actual time=4877..4877 rows=352 loops=1)
|> Aggregate using temporary table (actual time=4877..4877 rows=352 loops=1)
|> Nested loop inner join (cost=-398993 rows=595891) (actual time=31.6..1189 rows=599013 loops=1)
|> Filter: (f.dest_airport_id is not null) (cost=62740 rows=595891) (actual time=30.4..852 rows=599013 loops=1)
|> Covering index scan on f using idx_g2_designa_origin_dest (cost=62740 rows=595891) (actual time=30.4..785 rows=599013 loops=1)
|> Single-row index lookup on a using PRIMARY (airport_id=f.dest_airport_id) (cost=0.464 rows=1) (actual time=255e-6..299e-6 rows=1 loops=599013)
+-----+
1 row in set (9.06 sec)
```

Design B: composite index on fact_flight(dest_airport_id, origin_airport_id)

```
mysql> EXPLAIN ANALYZE
-> SELECT *
-> FROM (
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Origin' AS role
->    FROM fact_flight f USE INDEX (idx_q2_designB_dest_origin)
->    JOIN dim_airport a ON f.origin_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
->  UNION
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Destination' AS role
->    FROM fact_flight f USE INDEX (idx_q2_designB_dest_origin)
->    JOIN dim_airport a ON f.dest_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
-> ) q;

+-----+
| EXPLAIN |
+-----+

+-----+
| -> Table scan on q (cost=2.5..2.5 rows=0) (actual time=8819..8819 rows=30 loops=1)
| ->   Union materialize with deduplication (cost=0..0 rows=0) (actual time=8819..8819 rows=30 loops=1)
| ->     Limit: 15 row(s) (actual time=4953..4953 rows=15 loops=1)
| ->       Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4953..4953 rows=15 loops=1)
| ->         Table scan on <temporary> (actual time=4953..4953 rows=352 loops=1)
| ->           Aggregate using temporary table (actual time=4953..4953 rows=352 loops=1)
| ->             > Nested loop inner join (cost=335244 rows=595891) (actual time=120..1633 rows=599013 loops=1)
| ->               > Filter: (f.origin_airport_id is not null) (cost=62837 rows=595891) (actual time=120..1332 rows=599013 loops=1)
| ->                 > Covering index scan on f using idx_q2_designB_dest_origin (cost=62837 rows=595891) (actual time=120..1291 rows=599013 loops=1)
| ->                   > Single-row index lookup on a using PRIMARY (airport_id=f.origin_airport_id) (cost=0.357 rows=1) (actual time=222a-6..259a-6 rows=1 loops=599013)
| ->             > Limit: 15 row(s) (actual time=3865..3865 rows=15 loops=1)
| ->               > Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=3865..3865 rows=15 loops=1)
| ->                 > Table scan on <temporary> (actual time=3865..3865 rows=352 loops=1)
| ->                   > Aggregate using temporary table (actual time=3865..3865 rows=352 loops=1)
| ->                     > Nested loop inner join (cost=335244 rows=595891) (actual time=76.3..1195 rows=599013 loops=1)
| ->                       > Filter: (f.dest_airport_id is not null) (cost=62837 rows=595891) (actual time=76.2..921 rows=599013 loops=1)
| ->                         > Covering index scan on f using idx_q2_designB_dest_origin (cost=62837 rows=595891) (actual time=76.2..859 rows=599013 loops=1)
| ->                           > Single-row index lookup on a using PRIMARY (airport_id=f.dest_airport_id) (cost=0.357 rows=1) (actual time=179a-6..211a-6 rows=1 loops=599013)
| ->
|
+-----+
1 row in set (8.82 sec)
```

Design C: two single-column indexes on fact_flight(origin_airport_id) and fact_flight(dest_airport_id)

```
mysql> EXPLAIN ANALYZE
-> SELECT *
-> FROM (
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Origin' AS role
->    FROM fact_flight f USE INDEX (idx_q2_designC_origin_only)
->    JOIN dim_airport a ON f.origin_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
->  UNION
->   (SELECT a.airport, a.display_airport_name,
->    COUNT(*) AS flight_count, 'Destination' AS role
->    FROM fact_flight f USE INDEX (idx_q2_designC_dest_only)
->    JOIN dim_airport a ON f.dest_airport_id = a.airport_id
->    GROUP BY a.airport, a.display_airport_name
->    ORDER BY flight_count DESC
->    LIMIT 15)
-> ) q;

+-----+
| EXPLAIN |
+-----+

+-----+
| -> Table scan on q (cost=2.5..2.5 rows=0) (actual time=8348..8348 rows=30 loops=1)
| ->   Union materialize with deduplication (cost=0..0 rows=0) (actual time=8348..8348 rows=30 loops=1)
| ->     Limit: 15 row(s) (actual time=4317..4317 rows=15 loops=1)
| ->       Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4317..4317 rows=15 loops=1)
| ->         Table scan on <temporary> (actual time=4317..4317 rows=352 loops=1)
| ->           Aggregate using temporary table (actual time=4317..4317 rows=352 loops=1)
| ->             > Nested loop inner join (cost=335331 rows=595891) (actual time=127..1644 rows=599013 loops=1)
| ->               > Filter: (f.origin_airport_id is not null) (cost=62924 rows=595891) (actual time=127..1367 rows=599013 loops=1)
| ->                 > Covering index scan on f using idx_q2_designC_origin_only (cost=62924 rows=595891) (actual time=127..1308 rows=599013 loops=1)
| ->                   > Single-row index lookup on a using PRIMARY (airport_id=f.origin_airport_id) (cost=0.357 rows=1) (actual time=185a-6..216a-6 rows=1 loops=599013)
| ->             > Limit: 15 row(s) (actual time=4031..4031 rows=15 loops=1)
| ->               > Sort: flight_count DESC, limit input to 15 row(s) per chunk (actual time=4031..4031 rows=15 loops=1)
| ->                 > Table scan on <temporary> (actual time=4031..4031 rows=352 loops=1)
| ->                   > Aggregate using temporary table (actual time=4031..4031 rows=352 loops=1)
| ->                     > Nested loop inner join (cost=335331 rows=595891) (actual time=19.9..1398 rows=599013 loops=1)
| ->                       > Filter: (f.dest_airport_id is not null) (cost=62924 rows=595891) (actual time=19.9..1123 rows=599013 loops=1)
| ->                         > Covering index scan on f using idx_q2_designC_dest_only (cost=62924 rows=595891) (actual time=19.9..1062 rows=599013 loops=1)
| ->                           > Single-row index lookup on a using PRIMARY (airport_id=f.dest_airport_id) (cost=0.357 rows=1) (actual time=177a-6..214a-6 rows=1 loops=599013)
| ->
|
+-----+
1 row in set (8.36 sec)
```


Analysis:

For Query 2, there were much clearer cost improvements from indexing than query 1. The baseline had a dominant nested loop join cost of 717,870. Design A, using the composite index (origin_airport_id, dest_airport_id), reduced this cost to 398,993. Design B, using (dest_airport_id, origin_airport_id), reduced it to 335,244, which was the lowest across all the configurations we tested. Design C, using separate single-column indexes on origin_airport_id and dest_airport_id, resulted in a cost of 335,331. So Design B was the best-performing configuration.

Even with these reductions, the query still had to aggregate nearly the entire fact_flight table in both branches of the UNION, materialize temporary tables, and sort the grouped airport counts before returning the top 15 results. So, even though the indexing helped the access path into fact_flight, it couldn't eliminate the heavy grouping and sorting work required by the query, so the cost gains were noticeable but not too dramatic. We selected design B as the final design for Query 2 since it produced the lowest optimizer cost.

Query 3: Baseline:

```
--> SELECT origin.airport AS origin_code,
-->        dest.airport AS dest_code,
-->        dt.delay_type name,
-->        ROUND(SUM(fd.delay_minutes), 2) AS total_delay_minutes,
-->        COUNT(*) AS delay_occurrences
--> FROM fact_flight_delay fd
--> JOIN fact_flight f FORCE INDEX (PRIMARY)
--> ON fd.flight_id = f.flight_id
--> JOIN dim_airport origin ON f.origin_airport_id = origin.airport_id
--> JOIN dim_airport dest ON f.dest_airport_id = dest.airport_id
--> JOIN dim_delay_type dt ON fd.delay_type_id = dt.delay_type_id
--> JOIN (
-->   SELECT origin_airport_id, dest_airport_id
-->   FROM fact_flight FORCE INDEX (PRIMARY)
-->   WHERE arr_del15 = 1
-->   GROUP BY origin_airport_id, dest_airport_id
-->   ORDER BY COUNT(*) DESC
-->   LIMIT 10
--> ) sub
--> ON f.origin_airport_id = sub.origin_airport_id
--> AND f.dest_airport_id = sub.dest_airport_id
--> GROUP BY origin.airport, dest.airport, dt.delay_type_name
--> ORDER BY total_delay_minutes DESC
--> LIMIT 15;

1 row in set (13.14 sec)
```

Design A: composite index on fact_flight(arr_del15, origin_airport_id, dest_airport_id)

```
mysql> EXPLAIN ANALYZE
--> SELECT origin.airport AS origin_code,
-->        dest.airport AS dest_code,
-->        dt.delay_type name,
-->        ROUND(SUM(fd.delay_minutes), 2) AS total_delay_minutes,
-->        COUNT(*) AS delay_occurrences
--> FROM fact_flight_delay fd
--> JOIN fact_flight f USE INDEX (idx_q3_designA_del15_origin_dest)
--> ON fd.flight_id = f.flight_id
--> JOIN dim_airport origin ON f.origin_airport_id = origin.airport_id
--> JOIN dim_airport dest ON f.dest_airport_id = dest.airport_id
--> JOIN dim_delay_type dt ON fd.delay_type_id = dt.delay_type_id
--> JOIN (
-->   SELECT origin_airport_id, dest_airport_id
-->   FROM fact_flight USE INDEX (idx_q3_designA_del15_origin_dest)
-->   WHERE arr_del15 = 1
-->   GROUP BY origin_airport_id, dest_airport_id
-->   ORDER BY COUNT(*) DESC
-->   LIMIT 10
--> ) sub
--> ON f.origin_airport_id = sub.origin_airport_id
--> AND f.dest_airport_id = sub.dest_airport_id
--> GROUP BY origin.airport, dest.airport, dt.delay_type_name
--> ORDER BY total_delay_minutes DESC
--> LIMIT 15;

1 row in set (2.61 sec)
```

Design B: composite index on fact_flight(origin_airport_id, dest_airport_id, arr_del15)

```
mysql> EXPLAIN ANALYZE
-> SELECT origin.airport AS origin_code,
->        dest.airport AS dest_code,
->        dt.delay_type name,
->        ROUND(SUM(fd.delay_minutes), 2) AS total_delay_minutes,
->        COUNT(*) AS delay_occurrences
-> FROM fact_flight_delay fd
-> JOIN fact_flight f USE INDEX (idx_q3 designB origin_dest_del15)
->   ON fd.flight_id = f.flight_id
-> JOIN dim_airport origin ON f.origin_airport_id = origin.airport_id
-> JOIN dim_airport dest ON f.dest_airport_id = dest.airport_id
-> JOIN dim_delay_type dt ON fd.delay_type_id = dt.delay_type_id
-> JOIN (
->   SELECT origin_airport_id, dest_airport_id
->   FROM fact_flight USE INDEX (idx_q3 designB origin_dest_del15)
->   WHERE arr_del15 = 1
->   GROUP BY origin_airport_id, dest_airport_id
->   ORDER BY COUNT(*) DESC
->   LIMIT 10
-> ) sub
-> ON f.origin_airport_id = sub.origin_airport_id
-> AND f.dest_airport_id = sub.dest_airport_id
-> GROUP BY origin.airport, dest.airport, dt.delay_type_name
-> ORDER BY total_delay_minutes DESC
-> LIMIT 15;

1 row in set (1.53 sec)
```

1 -> Limit: 15 row(s) (actual time=1827..1827 rows=15 loops=1)
-> Sort: total_delay_minutes DESC, limit input to 15 row(s) per chunk (actual time=1827..1827 rows=15 loops=1)
-> Table scan on temporary (actual time=1821..1827 rows=4 loops=1)
-> Aggregate using temporary table (actual time=1827..1827 rows=4 loops=1)
-> Nested loop inner join (cost=2173 rows=1859) (actual time=1421..1820 rows=2668 loops=1)
-> Nested loop inner join (cost=1488 rows=1958) (actual time=1421..1814 rows=2668 loops=1)
-> Nested loop inner join (cost=123 rows=885) (actual time=906..951 rows=6957 loops=1)
-> Nested loop inner join (cost=23 rows=10) (actual time=905..937 rows=10 loops=1)
-> Nested loop inner join (cost=12.8 rows=10) (actual time=905..934 rows=10 loops=1)
-> Filter: (sub.origin_airport_id is not null) and (sub.dest_airport_id is not null) (cost=0.362..3.62 rows=10) (actual time=905..905 rows=10 loops=1)
-> Table scan on sub (cost=2.1..2.1 rows=0) (actual time=901..903 rows=10 loops=1)
-> Materialize (cost=0..0 rows=0) (actual time=903..905 rows=10 loops=1)
-> Limit: 10 row(s) (actual time=905..905 rows=10 loops=1)
-> Sort: 'count(*)' DESC, limit input to 10 row(s) per chunk (actual time=905..905 rows=10 loops=1)
-> Stream results (cost=122364 rows=7511) (actual time=941..963 rows=5718 loops=1)
-> Group aggregate: count(*) (cost=122364 rows=7111) (actual time=941..963 rows=5718 loops=1)
-> Filter: (fact_flight.arr_del15 = 1) (cost=62775 rows=98991) (actual time=941..989 rows=109605 loops=1)
-> Covering index lookup on fact_flight using idx_q3 designB origin_dest_del15 (cost=62775 rows=98991) (actual time=941..981 rows=99013 loops=1)
-> Single-row index lookup on origin using PRIMARY (airport_id=sub.origin_airport_id) (cost=0.026 rows=1) (actual time=937..937 rows=1 loops=10)
-> Single-row index lookup on dest using PRIMARY (airport_id=sub.dest_airport_id) (cost=0.026 rows=1) (actual time=0.319..0.319 rows=1 loops=10)
-> Covering index lookup on f using idx_q3 designB origin_dest_del15 (origin_airport_id=sub.origin_airport_id, dest_airport_id=sub.dest_airport_id) (cost=2.13 rows=86.5) (actual time=0.559..1.33 rows=696 loops=1)
-> Index lookup on fd using PRIMARY (flight_id=f.flight_id) (cost=1.31 rows=2.21) (actual time=0.124..0.124 rows=0.383 loops=6957)
-> Single-row index lookup on dt using PRIMARY (delay_type_id=fd.delay_type_id) (cost=0.25 rows=1) (actual time=0.00202..0.00206 rows=1 loops=2668)

Design C: composite index on fact_flight_delay(delay_type_id, flight_id)

```
mysql> EXPLAIN ANALYZE
-> SELECT origin.airport AS origin_code,
->        dest.airport AS dest_code,
->        dt.delay_type name,
->        ROUND(SUM(fd.delay_minutes), 2) AS total_delay_minutes,
->        COUNT(*) AS delay_occurrences
-> FROM fact_flight_delay fd USE INDEX (idx_q3 designC delaytype_flight)
-> JOIN fact_flight f USE INDEX (idx_q3 designA del15_origin_dest)
->   ON fd.flight_id = f.flight_id
-> JOIN dim_airport origin ON f.origin_airport_id = origin.airport_id
-> JOIN dim_airport dest ON f.dest_airport_id = dest.airport_id
-> JOIN dim_delay_type dt ON fd.delay_type_id = dt.delay_type_id
-> JOIN (
->   SELECT origin_airport_id, dest_airport_id
->   FROM fact_flight USE INDEX (idx_q3 designA del15_origin_dest)
->   WHERE arr_del15 = 1
->   GROUP BY origin_airport_id, dest_airport_id
->   ORDER BY COUNT(*) DESC
->   LIMIT 10
-> ) sub
-> ON f.origin_airport_id = sub.origin_airport_id
-> AND f.dest_airport_id = sub.dest_airport_id
-> GROUP BY origin.airport, dest.airport, dt.delay_type_name
-> ORDER BY total_delay_minutes DESC
-> LIMIT 15;

1 row in set (1.64 sec)
```

1 -> Limit: 15 row(s) (actual time=1531..1531 rows=15 loops=1)
-> Sort: total_delay_minutes DESC, limit input to 15 row(s) per chunk (actual time=1531..1531 rows=15 loops=1)
-> Table scan on temporary (actual time=1531..1531 rows=4 loops=1)
-> Aggregate using temporary table (actual time=1531..1531 rows=4 loops=1)
-> Nested loop inner join (cost=399472 rows=254) (actual time=1321..1526 rows=2668 loops=1)
-> Inner hash join (no condition) (cost=599293 rows=254) (actual time=1011..1520 rows=24785 loops=1)
-> Table scan on dt (cost=0.0149 rows=5) (actual time=0.0263..0.0527 rows=5 loops=1)
-> Hash
-> Inner hash join (f.dest_airport_id = sub.dest_airport_id), (f.origin_airport_id = sub.origin_airport_id) (cost=599293 rows=50.7) (actual time=300..1007 rows=6957 loops=1)
-> Hash
-> Nested loop inner join (cost=22.6 rows=10) (actual time=158..158 rows=10 loops=1)
-> Nested loop inner join (cost=13.1 rows=10) (actual time=158..158 rows=10 loops=1)
-> Filter: (sub.origin_airport_id is not null) and (sub.dest_airport_id is not null) (cost=0.362..3.62 rows=10) (actual time=158..158 rows=10 loops=1)
-> Table scan on sub (cost=2.1..2.1 rows=0) (actual time=158..158 rows=10 loops=1)
-> Materialize (cost=0..0 rows=0) (actual time=158..158 rows=10 loops=1)
-> Limit: 10 row(s) (actual time=158..158 rows=10 loops=1)
-> Sort: 'count(*)' DESC, limit input to 10 row(s) per chunk (actual time=158..158 rows=10 loops=1)
-> Stream results (cost=43370 rows=7511) (actual time=0.636..157 rows=5718 loops=1)
-> Group aggregate: count(*) (cost=43370 rows=7111) (actual time=0.633..156 rows=5718 loops=1)
-> Covering index lookup on fact_flight using idx_q3 designA del15 origin_dest (arr_del15=1) (cost=21985 rows=213948) (actual time=0.62..143 rows=109605 loops=1)
-> Single-row index lookup on origin using PRIMARY (airport_id=sub.origin_airport_id) (cost=0.057 rows=1) (actual time=0.00885..0.00888 rows=1 loops=10)
-> Single-row index lookup on dest using PRIMARY (airport_id=sub.dest_airport_id) (cost=0.057 rows=1) (actual time=0.00424..0.00428 rows=1 loops=10)
-> Single-row index lookup on fd using idx_q3 designC delaytype_flight (delay_type_id=dt.delay_type_id, flight_id=f.flight_id) (cost=0.498 rows=1) (actual time=0.0144..0.0144 rows=0.0767 loops=34785)

Analysis:

For Query 3, in the baseline version, the optimizer had to perform a full scan of fact_flight inside the derived table that identifies the top delayed routes, with the dominant join portion of the plan having a cost of 560,669, and the route-selection subquery including a fact_flight table scan cost of 652,393. After indexing, the costs dropped a lot. Design A used (arr_del15, origin_airport_id, dest_airport_id) which enabled a covering index lookup for delayed flights and reduced the subquery cost to 43,370, but the overall top join cost in that plan was still high at 598,876. Design B used (origin_airport_id, dest_airport_id, arr_del15) which gave us the lowest overall dominant join cost at 2,173. Design C added the fact_flight_delay(delay_type_id, flight_id) index while using Design A's fact-flight index, and it produced a dominant join cost of 599,472, which was not as strong as Design B.

These results show that Query 3 benefits most from indexing the route structure used by the rewritten join-based query. Design B aligned well with the join conditions on (origin_airport_id, dest_airport_id) while still including arr_del15 in the index, which allowed mysql to do efficient route-based lookups after finding the top delayed routes. Design B gave the best total optimizer cost for the full query plan, so we chose it as the final design for Query 3.